

Automated formal analysis of Signal's Double Ratchet: Attacks, Fixes and Security Proofs

Vincent Cheval¹, Charlie Jacomme², **Jessica Richards**¹

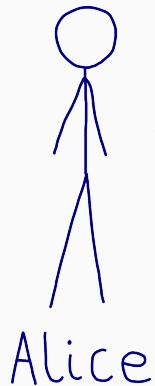
3rd June 2026

¹ University of Oxford

² Université de Lorraine, CNRS, Inria, LORIA

- Protocol for end-to-end encryption of messages
- Used in Signal, WhatsApp and Facebook Messenger
- Three main protocols
 - Post Quantum eXtended Diffie-Hellman (PQXDH)
 - Double Ratchet (DR)
 - Sesame
- PQXDH provides initial key
- Double Ratchet sends / receives messages
- Sesame manages sessions

Double Ratchet Introduction

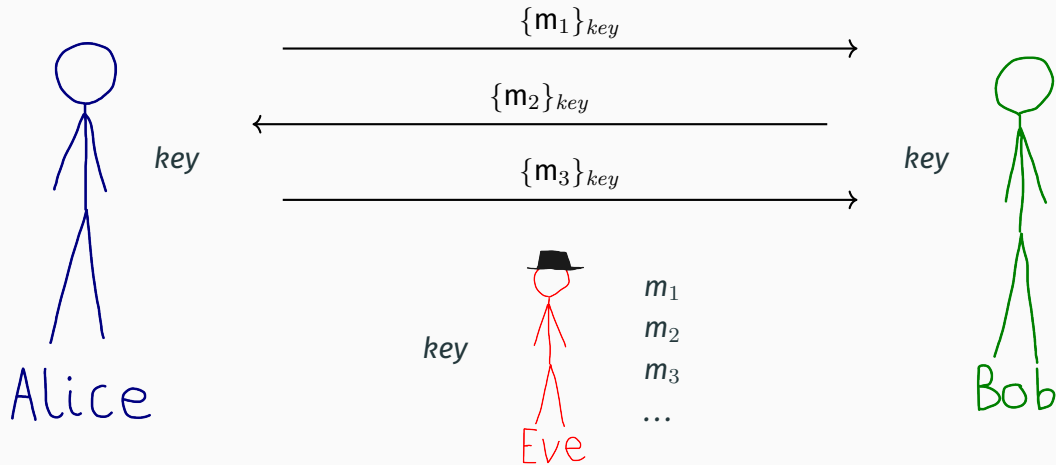


key

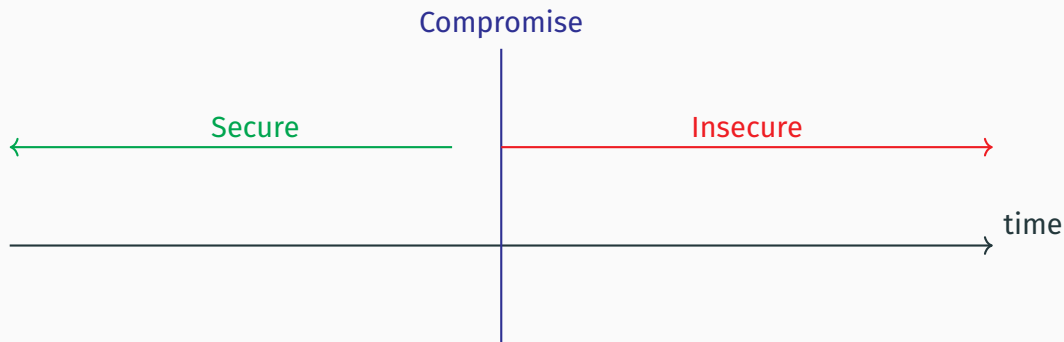


key

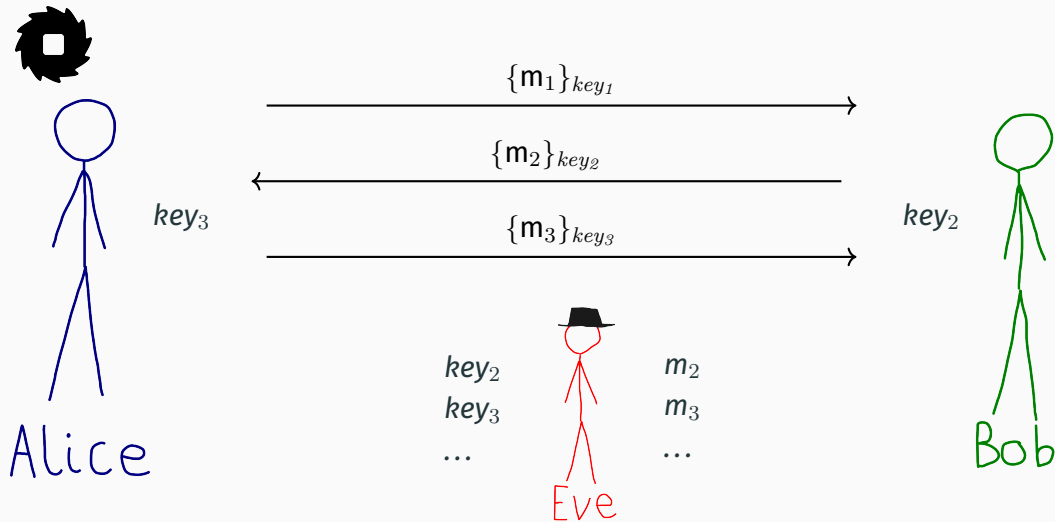
No Ratchet



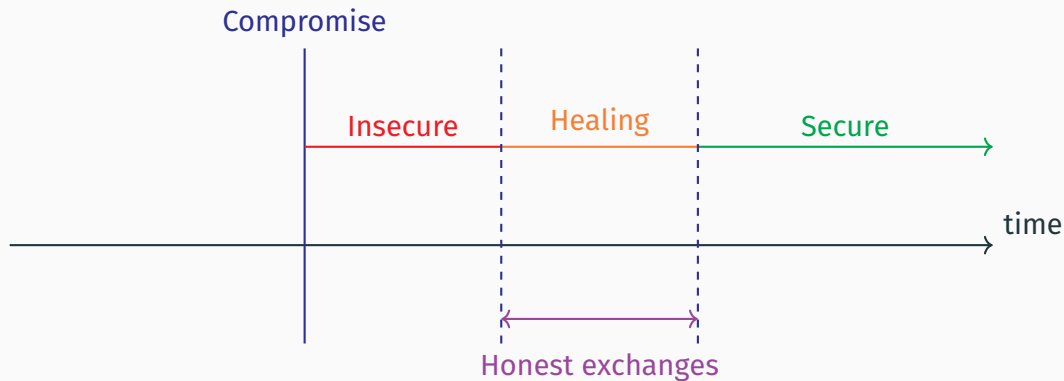
Forward secrecy (FS)



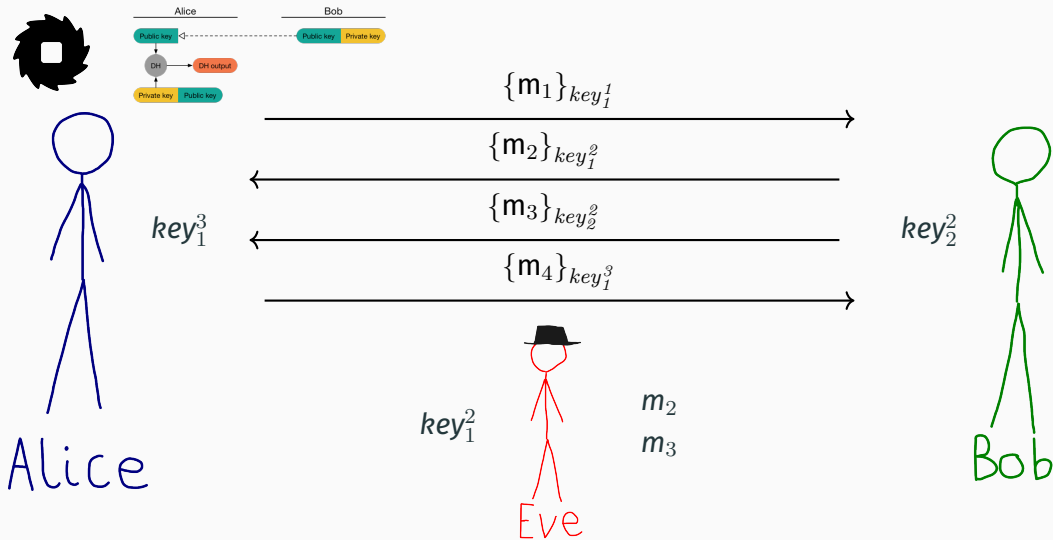
Single Ratchet



Post compromise security (PCS)

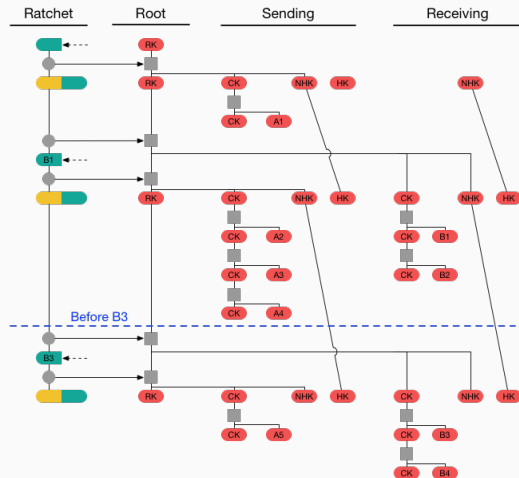


Double Ratchet



Full Double Ratchet

- More layers of keys
- Sending and receiving chains
- PQXDH creates an initial root key
- Skipped messages
- Header encryption¹



¹Not implemented in Signal

Our analysis improves upon prior DR analyses by including all of the following:

- Fully autonomous proofs of FS and a strong form of PCS in ProVerif
- Covers the entire DR specification including multiple sessions and skipped messages
- Extra variants analysed such as header encryption, an insecure SPQR and composition with PQXDH
- Discovered three previously unknown attacks on FS
 - Weak State Attack
 - Replay Attack via Key Mangling
 - None Key Attack

- Automated protocol verification tool
- Specify protocols and security properties
- Proves security properties in the Symbolic Model
- May not terminate or may yield false attacks.



Main challenge: Creating a complete faithful model of DR which ProVerif can prove secure.

Encoding Double Ratchet

```
def RatchetEncrypt(state, plaintext, AD):  
    state.CKs, mk = KDF_CK(state.CKs)  
    header = HEADER(state.DHs, state.PN, state.Ns)  
    state.Ns += 1  
    return header, ENCRYPT(mk, plaintext, CONCAT(AD, header))
```



```
let RatchetEncrypt(sinfo:session_info, plaintext:bitstring, ad:associated_data) =  
    in(state_chan(sinfo), cell(st,m_idx,r_idx));  
  
    let (cks:chain_key,mk:message_key) = kdf_ck(CKs(st)) in  
    let hd = header( DHs(st), PN(st),  Ns(st)) in  
    let new_st = state(DHs(st),DHR(st),RK(st),cks,CKr(st),Ns(st)+1,Nr(st),PN(st)) in  
    out(att, (hd, encrypt(mk, plaintext, concat(ad,hd))));  
  
    out(state_chan(sinfo), cell(new_state,m_idx+1,r_idx))  
.
```



ProVerif example - actual

```
let RatchetEncrypt(sinfo:session_info, plaintext:bitstring, ad:associated_data) =
  in(state_chan(sinfo), cell(st,m_idx,r_idx, tr));

  (** Proof Helper **)
  event Sanity(state_chan(sinfo),0,m_idx,r_idx);
  let () = PH_counter_change(modif_counter(sinfo),m_idx) in
  let () = PH_synch_value(modif_counter(sinfo),m_idx,(st,r_idx, tr)) in
  (** End Proof Helper **)

if CKs(st) <> none_ck then
let (cks:chain_key,mk:message_key) = kdf_ck(CKs(st)) in
let hd = header( DHs(st), PN(st), Ns(st)) in
let KeyPair(_,dhs_pub) = DHs(st) in
event MessageKey(sinfo,Sender,dhs_pub,mk, Ns(st), r_idx);
event Send(sinfo, dhs_pub, Ns(st), plaintext);
let new_state = state(DHs(st), DHr(st), RK(st), cks, CKr(st), Ns(st) + 1, Nr(st), PN(st)) in
out(att, (hd, encrypt(mk,plaintext, concat(ad,hd))));

out(state_chan(sinfo), cell(new_state,m_idx+1,r_idx, tr))
.
```

Encoding loops

Function for out-of-order messages

```
def SkipMessageKeys(state, until):  
    if state.Nr + MAX_SKIP < until:  
        raise Error()  
    if state.CKr != None:  
        while state.Nr < until:  
            state.CKr, mk = KDF_CK(state.CKr)  
            state.MKSKIPPED[state.DHr, state.Nr] = mk  
            state.Nr += 1
```

Encoding Loops - loop body

```
let SkipMessageKeys =  
  in(st_chan, cell(st,r_idx,m_idx,rcv_idx));  
  
let (ckr:chain_key,mk:message_key) = kdf_ck(CKr(st)) in  
insert_skipped_message(sinfo, DHr(st), Nr(st), mk, m_idx, r_idx) in  
let new_st = update_ckr(st, ckr) in  
  
out(st_chan, cell(new_st,r_idx,m_idx+1,rcv_idx+1))  
.
```

Loop can run unboundedly when state is available.

Encoding loops - Usage

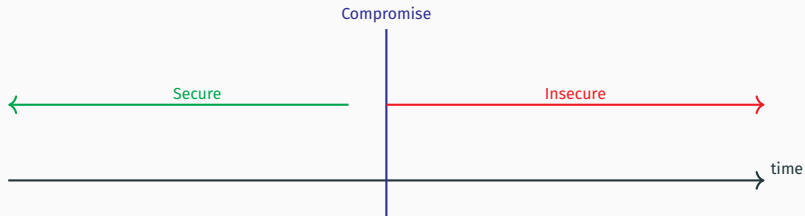
We need to skip all message keys between our current one and the one of our received message. Two cases that use this:

```
def RatchetDecrypt(state, header, ciphertext, AD):
    plaintext = TrySkippedMessageKeys(state, header, ciphertext, AD)
    if plaintext != None:
        return plaintext
    if header.dh != state.DHr:
        SkipMessageKeys(state, header.pn)
        DHRatchet(state, header)
    SkipMessageKeys(state, header.n)
    state.CKr, mk = KDF_CK(state.CKr)
    state.Nr += 1
    return DECRYPT(mk, ciphertext, CONCAT(AD, header))
```


Encoding Loops

```
let RatchetDecrypt =  
  in(att, (hd:header, ciphertext:bitstring));  
  in(st_chan, cell(st, r_idx, m_idx, rcv_idx));  
  
  TrySkippedMessages(sinfo, st, hd, ciphertext, ad)  
| ( if HeaderDHP <> DHr(st)  
  then (  
    if Nr(st) = headerPN then (* *)  
    let st1 = DHRatchet(st, hd) in  
  
    out(st_chan, cell(st1, r_idx+2, m_idx+1, rcv_idx)) | (* *)  
    in(st_chan, new_cell:cell_content);  
  
    if only_skipped_message_keys(cell(st1, r_idx+2, m_idx+1, rcv_idx), new_cell) then (* *)  
  
    BasicDecrypt(sinfo, hd, ciphertext, new_cell)  
  ) else (  
    BasicDecrypt(sinfo, hd, ciphertext, cell(st, r_idx, m_idx, rcv_idx))  
  )  
)
```

Encoding Forward Secrecy



```
query sd:session_id, idA,idB:id, n,ns,nr,r_idx,r_idx':nat, dhs:point, mk:message_key, ck:chain_key, rk:root_key;  
event(MessageKey(session(sd,idA,idB),Sender,dhs, mk,n,r_idx)) && attacker(mk) ==>
```

```
event(CompromiseMK(session(sd,idB,idA), mk,n,r_idx+1)) ||
```

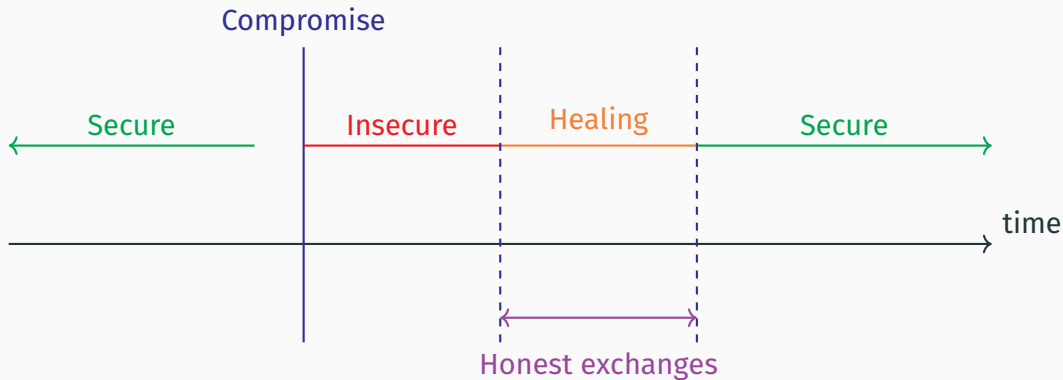
```
(event(CompromiseRK(session(sd,idA,idB),rk,r_idx')) && r_idx' <= r_idx) ||
```

```
(event(CompromiseRK(session(sd,idB,idA),rk,r_idx')) && r_idx' <= r_idx) ||
```

```
(event(CompromiseCKs(session(sd,idA,idB),ck,ns,r_idx)) && ns <= n ) ||
```

```
(event(CompromiseCKr(session(sd,idB,idA),ck,nr,r_idx+1)) && nr <= n)
```

Encoding Post Compromise Security



:(looks hard

Encoding Post Compromise Security

Same approach, assuming:

1. The attacker has learnt a message key in ratchet r
2. AND there is a prior r_{heal} -th ratchet associated with an honest DH key (not necessarily intended for the corresponding ratchet)

Then one of the following must have occurred:

1. Any of the compromises on the message keys or chain keys listed in the FS query
2. A root key was compromised after the healing ratchet (essentially FS)
3. *A root key was compromised before the healing ratchet, and one of the DH keys of the healing ratchet were compromised*

Encoding Post Compromise Security

```
query ...
  let sinfoS = session(sd,idA,idB) in
  let sinfoR = session(sd,idB,idA) in
  event(MessageKey(sinfoS,Sender,dhs, mk,n,r_idx)) && attacker(mk) &&
  mess(state_chan(sinfoS,cell(st,r_idx_heal,_,_))) &&
  event(HonestDH(sinfoR,dhr_prv',dhr',r_idx')) &&
  DHr(st) = dhr' && r_idx_heal <= r_idx ==>
  ( (* FS CK / MK compromises *)
    event(CompromiseMK(sinfoR,dr_mk,n,r_idx+1)) && mk=kdf_hybrid(dr_mk,pq_mk) ||
    (event(CompromiseCKs(sinfoS,_,ns,r_idx)) && ns <= n ) ||
    (event(CompromiseCKr(sinfoR,_,nr,r_idx+1)) && nr <= n) ||
    (* RK compromises*)
    event(CompromiseRK(sinfo,_,r_idx_comp)) && (sinfo = sinfoS || sinfo = sinfoR) && (
      (* ... after the heal *)
      (r_idx_heal <= r_idx_comp < r_idx) ||
      (* ... or before the heal but one of DH keys of healing session is compromised *)
      (r_idx_comp < r_idx_heal &&
        (event(CompromiseDHs(sinfoR, dhr_prv', r_idx'+2))) ||
        (event(CompromiseDHs(sinfoS, dhs_prv', r_idx_heal))))))
```

Verification of properties

Assuming a fresh root key for each session:

Proved	DR	DR + HE	PQXDH + DR ¹
Sanity	✓ (7s)	✓ (5m)	✓ (38m)
Forward Secrecy	✓ (1m)	✓ (1h)	✓ (4h)
PCS	✓ (41m)	? (✓ Less compromises)	✓ (11h)

Difficulties in proving state lemmas.

Out of memory (> 300 GB)

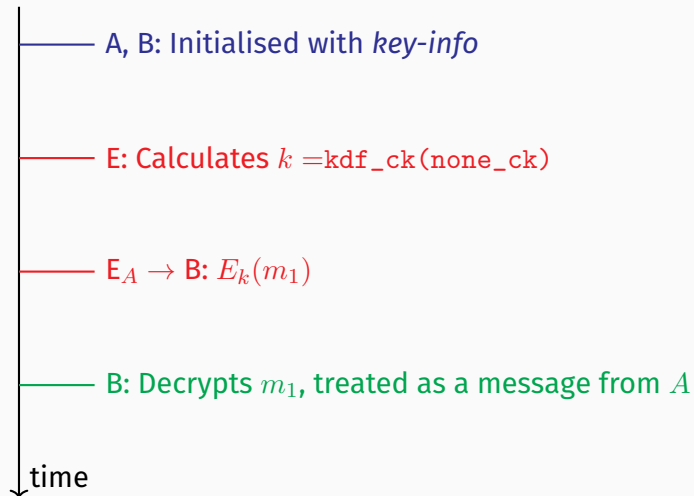
¹Limited attacker

None Value Attack / Weak States

- In PQXDH + DR, the first message is both PQXDH and DR.
- Both parties are able to send messages from the moment the session is set up.
- Standalone DR begins as $(\text{PQXDH} + \text{DR}) - \text{DR}$.
- Default values in many spaces
- *Weak state*

`state(SPK_B, none_dh, SK, none_ck, none_ck, 0, 0, 0)`

None Value Attack



- Could be an issue for people implementing their own versions of the DR.
- Just a matter of specification.

KDF_CK(ck): Returns a pair (32-byte chain key, 32-byte message key) as the output of applying a KDF keyed by a 32-byte chain key *ck* to some constant.

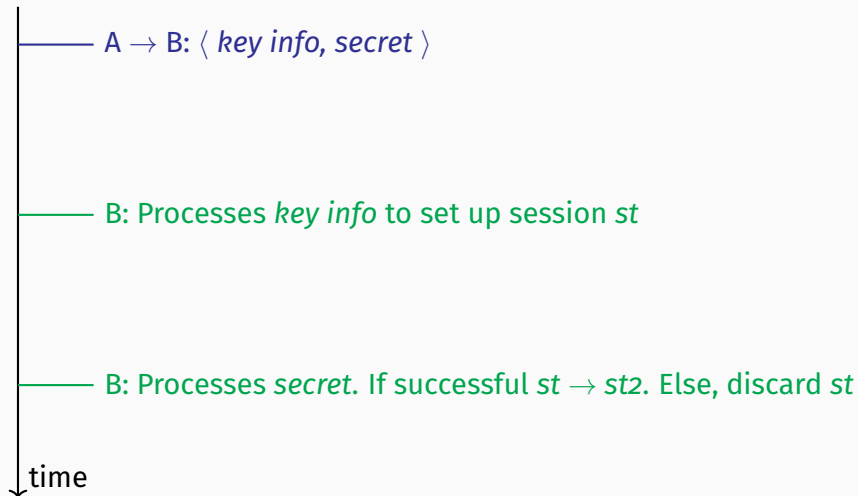
- Could be an issue for people implementing their own versions of the DR.
- Just a matter of specification.

KDF_CK(ck): Returns a pair (32-byte chain key, 32-byte message key) as the output of applying a KDF keyed by a 32-byte chain key *ck* to some constant. If *ck* is None this function must fail in a way that terminates processing.

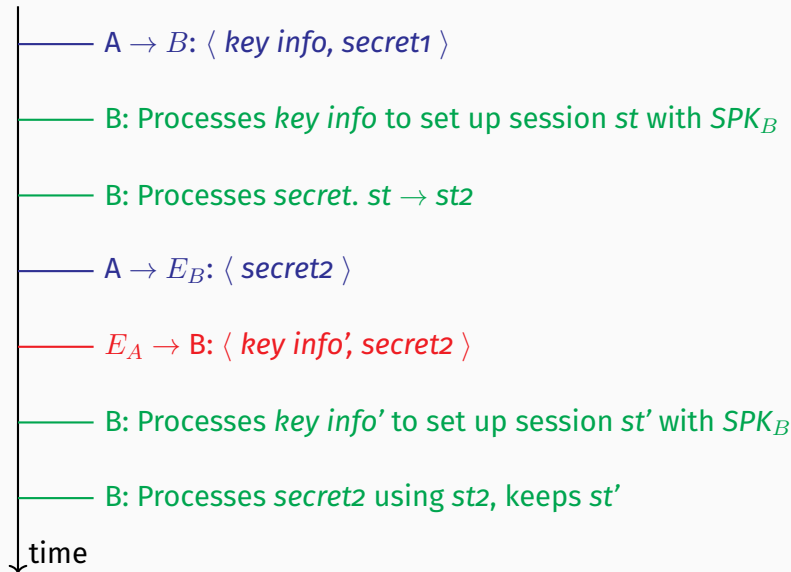
- Key reuse generally bad
- Keys stored in session memory should be short-lived keys
- Medium term key SPK_B of PQXDH features in the weak state

`state(SPK_B, none_dh, SK, none_ck, none_ck, 0, 0, 0)`

Session Set Up



Weak State Attack



- Practically executable attack
- Signed pre key can be recovered from memory after deletion
- Affects Forward Secrecy
- Fixed by ensuring that the first DR message is decrypted by the corresponding session
- Fixed by Signal in implementation²

²<https://github.com/signalapp/libsignal/commit/cd361186fba60c83186262d91b03af608fb8c7c3>

- Automated proof of FS and PCS for the Double Ratchet in ProVerif
- Some assurances for the combined protocols (Double Ratchet + PQXDH)
- Three Attacks found:
 1. Weak State Attack
 2. Replay Attack via key mangling
 3. Forward Secrecy Attack in the DR specification
- Fixes for (1, 2) in the implementation³. (3) fixed in specification.

³<https://github.com/signalapp/libsignal/commit/5293caa6cad81566b80f3c3731857bedaa78737b>